

Jigsaw: Retrieval-Constrained Exact Subgraph Matching

Abstract. Exact subgraph-isomorphism solvers such as Glasgow are sound and complete but search a whole-graph candidate domain. On full OGBN-MAG (1.9M nodes), a direct CP-based Glasgow call solves **0 of 15** sampled queries; it is already infeasible on OGBN-Arxiv (169K, 0/15) yet trivial on Cora (19.8K, 15/15 in ≈ 2.7 s). We introduce **Jigsaw**, a retrieval-constrained verifier: retrieve hierarchical partitions, stitch one-hop overlap, prune by typed signatures and exact labels, then invoke Glasgow. Jigsaw is deliberately *retriever-agnostic*. Its GNN uses **FullCov**, a coverage objective requiring every touched partition (locked Arxiv K-hop: FullCov@100 66.0% $\rightarrow$ 82.0%, McNemar $p=4.0\times 10^{-10}$), but the strongest bounded retriever depends on *label selectivity*: a classical label index dominates under near-unique labels (96.7% > learned 88.6%), while learned retrieval becomes the better localizer as labels coarsen. Across Cora, Arxiv, and MAG, Jigsaw makes most MAG queries exactly solvable (88.6% learned, 98.4% exhaustive), remains exact inside the retrieved candidate, rejects audited negatives at 99.7–100.0%, and streams MAG at 2.4 GB rather than 10.2 GB for whole-graph residence.

Keywords: Subgraph isomorphism · Graph neural networks · Exact verification · Neural retrieval

1 Introduction

Subgraph isomorphism is used in motif discovery, scientific graph analysis, fraud search, and security auditing. Exact solvers are sound and complete but pay for large candidate domains; learned retrieval scales better but does not certify matches. Jigsaw composes the two: learned narrowing followed by exact verification.

Jigsaw partitions a large graph, embeds queries and partitions into a shared space, retrieves candidate partitions, stitches overlap neighborhoods, prunes by typed signatures and exact labels, and calls Glasgow. If retrieval covers the planted match, the verifier certifies the result inside the candidate; if retrieval misses a required region, local soundness remains but global completeness is lost. FullCov makes this coverage condition explicit.

We evaluate Cora, OGBN-Arxiv, and MAG with two seeds, three sizes, six positive and two negative families, and six retrieval policies. Cora/Arxiv reach 100% only at the exhaustive all-partition budget; at the matched half-partition budget Jigsaw reaches 94.2/92.8/88.6% on Cora/Arxiv/MAG.

Contributions.

- We scale exact subgraph matching to MAG via retrieval-constrained verification: direct Glasgow solves 0/15 MAG queries, while Jigsaw streams a bounded candidate at 2.4 GB versus 10.2 GB for whole-graph residence.
- We separate verifier soundness from retrieval completeness, treat budget as a completeness knob, and prove pruning is coverage-lossless.
- We introduce **FullCov**, which optimizes the weakest required partition and is decisive when overlap cannot repair misses; the learned retriever is strongest under coarse labels, while a classical label index wins under near-unique labels.
- We provide a two-seed, three-dataset benchmark and a candidate-shrinkage analysis with selective-overlap operators that reduce admitted overlap volume by 3–6 $\times$.

2 Related Work

Exact solvers. Classical solvers such as VF2 [3], RI [8], LAD [9], VF3 [18], and Glasgow [1] use propagation, branching, and search ordering; filter-and-verify matchers such as TurboISO [15], CFL-Match [16], DAF [17], and GSI [19] first restrict query-vertex candidates and then enumerate embeddings [20]. Their filtering is the closest classical analogue to Jigsaw’s pruning, but the candidate domain remains whole-graph. Out-of-core and distributed matchers avoid residence by streaming or sharding the *entire* graph; Jigsaw instead bounds residence by region relevance. Direct CFL-Match/DP-iso/GQL runs solve Cora/Arxiv and high-selectivity MAG once loaded, but require residence and collapse at coarse labels (12/27); CP-based Glasgow never finishes full MAG (0/15). Jigsaw therefore claims a bounded-memory frontier, not faster resident matching.

Neural graph matching and retrieval. GNN encoders such as GIN [2] and GraphSAGE are natural graph retrievers. NeuroMatch [11], GLSearch [13], and neural subgraph counting [14] score alignments or counts directly. GNN-PE [12] prunes candidates for exact matching with path-dominance embeddings, but operates at node/path level and needs the whole graph/index resident. Jigsaw instead ranks coarse regions so an exact solver runs on a reduced domain; the central metric is partition coverage.

Approximate indexing. Jigsaw uses FAISS [4] for nearest-neighbor search over partition embeddings and METIS [5] for hierarchical graph partitioning. The offline preprocessing cost is intended for static graphs with many future queries, similar to database indexing.

3 Problem and Guarantee

Let $G = (V, E, X)$ be a large attributed graph and $Q = (V_Q, E_Q, X_Q)$ a query. The full exact task asks whether Q is isomorphic to an induced or non-induced

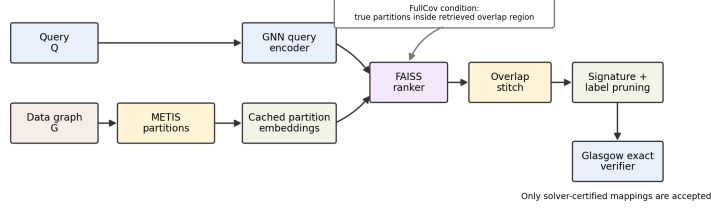


Fig. 1. Jigsaw pipeline. The learned component ranks partitions, but the only accepting step is Glasgow verification inside the stitched candidate graph. FullCov is the condition under which the candidate contains every partition needed by the planted match.

subgraph of G under the solver’s configured semantics. Jigsaw instead builds a candidate graph $G_C \subseteq G$ and runs the exact verifier on G_C .

The verifier is sound inside G_C : a returned match is a true match. Completeness is conditioned on coverage. Let $\mathcal{T}(Q)$ be the set of coarse partitions touched by the planted match and $R_K(Q)$ the retrieved top- K partition set. We define

$$\text{FullCov@K}(Q) = \mathbf{1}[\mathcal{T}(Q) \subseteq R_K(Q)]. \quad (1)$$

For overlap-cascade verification, R_K is expanded by one-hop partition overlap before pruning. FullCov is a sufficient retrieval condition for exact verification over planted positives; negative queries instead test whether the candidate-and-solver pipeline correctly returns no match.

Budget as a soundness-preserving completeness dial. Soundness never depends on K : any returned match is a true match for every budget, so K is a pure *completeness* control. At $K = |\mathcal{P}|$ (all partitions; FilterAll) the candidate is the whole graph and verification is complete; as K shrinks completeness degrades gracefully and *measurably* – the solved-by-budget curves of Section 6.2 are exactly the realized completeness as a function of K . The system thus exposes a tunable point on a recall–cost frontier rather than a fixed approximation: a caller needing certainty pays for $K = |\mathcal{P}|$, one needing speed accepts a quantified miss rate. This reframes the conditional guarantee as a knob with endpoints (exact-but-exhaustive vs. fast-but-incomplete) rather than a hidden failure mode.

4 Jigsaw Method

Hierarchical localization and overlap cascade. The data graph is partitioned into fine and coarse METIS blocks: fine blocks give local structure, coarse blocks are the retrieval units. For each query Jigsaw computes $z_Q = f_\theta(Q)$ and ranks all coarse partitions by cosine similarity to cached partition embeddings (FAISS provides fast top- K). Because hard partition boundaries are brittle – true query neighborhoods often cross coarse blocks – Jigsaw then stitches the retrieved partitions plus one-hop neighboring overlap, increasing recall before verification while still avoiding full-graph search.

Pruning and verification. The stitched region is filtered by typed structural signatures (matching coarse type and feature fingerprints) and exact labels (especially important for negative-label queries) before Glasgow verification. The final answer is produced only by the exact verifier. An operator-necessity ablation on a shared 48-query set shows these operators are *dataset-dependent*: on MAG, removing overlap halves the positive solve rate (88.9→44.4%) while signatures are inert; on Arxiv, overlap is a no-op while removing signatures explodes the candidate domain $\sim 75\times$ (94→7,081 nodes). Each is decisive on the graph that stresses it, so none can be dropped – overlap recovers boundary-crossing matches at scale, signatures prevent candidate explosion on label-rich graphs.

FullCov objective. Standard contrastive training rewards high average similarity to positives, which can still leave one required partition ranked too low; Jigsaw optimizes the bottleneck. For query Q_i , let $\mathcal{P}_i$ be its positive partitions and s_{ij} the query-partition score. The objective combines multi-positive contrastive learning with a weak-positive term:

$$\mathcal{L}_{\text{weak}}(i) = -\log \frac{\exp(\min_{p \in \mathcal{P}_i} s_{ip}/\tau)}{\exp(\min_{p \in \mathcal{P}_i} s_{ip}/\tau) + \sum_{n \in \mathcal{N}_i} \exp(s_{in}/\tau)}. \quad (2)$$

Additional CVaR and barrier terms penalize positives below the retrieval budget, and partition embeddings are refreshed from cache with live re-encoding of required positives so the weakest receives a usable gradient. The Arxiv ablation in Table 3 isolates this objective under matched training budgets.

Why the weakest positive. The min (and its CVaR relaxation) follows from the metric. $\text{FullCov@K}(Q) = \mathbf{1}[\mathcal{T}(Q) \subseteq R_K(Q)]$ is a *conjunction*: it succeeds only if *every* $p \in \mathcal{P}_i$ ranks within K , so failure is governed by the worst-ranked required partition $\min_p s_{ip}$, not the average positive score contrastive learning maximizes. Optimizing the average can raise mean recall while leaving one partition below budget – the failure exact verification cannot tolerate. CVaR (mean of the worst α -fraction) is a smooth surrogate for the non-differentiable min, spreading gradient over the hardest positives; hence in Table 3 the objective improves fixed-budget FullCov most at the cascade’s operating points, though average recall barely moves.

4.1 Encoder Architecture and Training Interface

The encoder is shared between query and partition graphs: features are projected to 256-d, passed through six residual message-passing blocks with LayerNorm, and read out by multi-layer mean/max/sum pooling into an L_2 -normalized 128-d vector. This matters because the problem is region localization, not node alignment: the model must place a query near *every* partition needed for verification, including the farthest true one. For Cora/Arxiv the block is GraphSAGE (SAGEConv), at least as strong as GIN at the budgets the cascade uses. OGBN-MAG is *heterogeneous* (paper/author/institution/field nodes, typed relations);

we flatten to a single index but *retain* node and relation types via a six-layer RGCN (RGCNConv, per-relation bases over $2R$ symmetrized relations). Since only paper nodes carry features, others receive node-type one-hot and per-relation-degree features, so the retriever reasons over typed structure.

Both encoders share one retrieval interface: cached fine and coarse partition embeddings are compared to query embeddings, and positive partitions can be live-reencoded during training so coverage gradients update required positives instead of treating the partition bank as stale constants. Training uses three signals – multi-positive contrastive learning toward all touched partitions, hard negatives that separate nearby incorrect partitions, and the FullCov coverage term on the weakest required positive. In the final MAG run the selected checkpoint is the validation-best FullCov model rather than the final epoch, which is why the bundle separates `mag_rgcn_best` production rows from `mag_rgcn_final` diagnostic rows.

Why exactness is conditional. The architecture does not make the retriever an exact solver; it only proposes a candidate boundary, and once G_C is constructed Glasgow gives exact answers for G_C . A positive query can fail for two reasons: the retrieval boundary does not contain the planted solution, or the candidate becomes too large and the solver times out – the benchmark records both via coverage diagnostics, timeout counts, and solved-by-budget curves. Negative queries test whether the candidate construction plus exact verifier returns no match when the generator intended an impossible query.

5 Experimental Protocol

Datasets. We evaluate Cora [10] (small enough for exact timing sanity checks), OGBN-Arxiv [6] (the main medium-scale benchmark), and the full heterogeneous OGBN-MAG (1,939,743 nodes across four node types joined by typed relations), which stresses the approach with both scale and relation-aware structure and is modeled relation-aware (RGCN over retained node/edge types), not collapsed to an attribute-only homogeneous graph.

Connected-positive protocol. All positive families are generated as *connected* planted subgraphs; the earlier disconnected multi-region artifact is removed from the canonical grid.

Label granularity. The verifier uses discrete labels from hashed full attributes ($\sigma(v) = \text{md5}(x_v) \bmod 10^6$), so matches are exact for hashed-label semantics. These highly selective labels make exact-label methods strong and make retrieval mostly a candidate-cost lever; Section 6.2 tests coarser 39-class labels.

Queries and retrieval policies. The matrix uses two query seeds, sizes 20/50/100, and 50 queries per type/size/seed. Positives are single, multi-fine, K-hop, degree K-hop, random-walk, and multi-coarse; negatives perturb labels or structure. Policies are learned Jigsaw, MeanFeat, Mean-RRF, TopoFeat, Random, and exhaustive FilterAll.

Table 1. Partition balance. Coarse partitions are ranked by retrieval; fine partitions support local overlap and bridge construction. Node counts are per partition.

Dataset	Level	Graph nodes	Parts	Min	Median	Mean	P90	Max
Cora	Coarse	19,793	20	962	986	989.6	1,018	1,019
Cora	Fine	19,793	100	190	197	197.9	204	205
Arxiv	Coarse	169,343	200	822	846	846.7	872	872
Arxiv	Fine	169,343	1,000	153	169	169.3	175	195
MAG	Coarse	1,939,743	2,000	876	969	969.9	999	999
MAG	Fine	1,939,743	10,000	170	194	194.0	200	210

Table 2. Overlap expansion scale. The operator adds boundary nodes of edge-sharing partitions, not whole neighboring partitions; the loose union bound is never materialized.

Dataset	Coarse parts	Partition nodes (med)	+Boundary overlap (med)	Expanded part (med / $\times$)	Neighbor parts med / max	Loose union bound (med)
Cora	20	986	534	1,533 / 1.6 $\times$	18 / 19	18,820
Arxiv	200	846	1,606	2,470 / 2.9 $\times$	135 / 173	115,004
MAG	2,000	969	4,252	5,228 / 5.4 $\times$	948 / 1,999	921,494

Partition and overlap scale. Tables 1 and 2 summarize the hierarchy. On MAG, a coarse partition has median 969 nodes and borders many partitions, but one-hop overlap admits only boundary nodes (median 4,252), not whole neighboring partitions. The union over large budgets can be broad, yet pruning leaves much smaller verifier candidates (Section 6.3).

Metrics. We report FullCov@K/recall for retrieval and exact-solve rate, no-match accuracy, false positives, timeouts, candidate size, and wall-clock components for verification.

Benchmark completion and data hygiene. Each dataset has 288 production rows (6 methods $\times$ 2 seeds $\times$ 8 families $\times$ 3 sizes); diagnostic variants are kept separate from canonical rows.

Implementation. Partitioning uses METIS, embeddings use FAISS, and Glasgow verifies pruned `vertexlabelledlad` candidates. Timings are per-query wall-clock on Google Cloud CPU_X_8 with a 30s verifier budget. Encoders, query seeds, hierarchies, and manifests will be released.

6 Results

6.1 FullCov Training Improves Retrieval

Table 3 gives the matched-budget Arxiv retrieval result. All rows use the same locked K-hop query seeds and 9,000 optimizer steps. The FullCov objective im-

Table 3. Matched Arxiv retrieval objective ablation. Fixed retrieval budgets, two locked query seeds, two training replicates for control/final.

Objective	FullCov@20	FullCov@50	FullCov@100	Mean worst rank
Contrastive control	21.7%	39.0%	66.0%	74.02
FullCov objective	26.0%	48.8%	82.0%	58.97
McNemar p	4.6×10^{-3}	9.8×10^{-6}	4.0×10^{-10}	–

proves the fixed-budget operating points most relevant to bounded exact verification.

The effect is not a change in feasibility: the locked K-hop suite has zero queries impossible at $K = 20, 50, 100$; average true coarse footprint is 6.37 and maximum footprint is 17. Misses are ranking errors, and the objective reduces them by explicitly optimizing the weakest required partition. Because the control and FullCov-aligned models share query seeds and training budget, the improvement is not explained by easier queries or longer optimization. The largest gain appears at FullCov@100, where the downstream exact verifier has enough room to benefit from better ordering but still cannot simply search the whole graph – exactly the operating point the cascade uses.

6.2 Three-Dataset Production Matrix

Table 4 reports exact planted-match recovery, correct no-match behavior, false positives, solver timeouts, candidate nodes, runtime, and verifier time.

At the half-partition budget (Table 4) Jigsaw solves 94.2/92.8/88.6% on Cora/Arxiv/MAG. The 100% Cora/Arxiv entries are the exhaustive ceiling, not a retrieval result; Mean-RRF is competitive there, and FeatureIndex is strongest on MAG under near-unique labels. MAG is the stress case: Jigsaw sends fewer candidate nodes than FilterAll (138K vs 444K) and has lower verifier time (82 vs 289 ms), while FilterAll remains the recall ceiling at 98.4%. The cascade also streams the graph: partition-wise pruning before edge materialization peaks at **2.4 GB** on a 54-query MAG grid versus **10.2 GB** for whole-graph residence ($4.2\times$), recovering 46/54 matches.

Label-selectivity crossover. Near-unique md5 labels flatter exact-label methods: a classical label-coverage index (**FeatureIndex**) is the strongest bounded retriever here (96.7%; median worst-true-partition rank 104 of 2,000) vs. the learned GNN (88.6%; rank ~ 900). At coarse 39-class labels this *inverts* on ranking: the learned model has better worst-true-partition ranks in each family, while complete solve runs are statistically tied (FeatureIndex 63.4%; learned variants 63–64%). Learned retrieval helps as labels stop being selective, but the current evidence is a localization win, not a solve-rate win, and the learned model is near-random on spatially-extended families (k-hop, random-walk; rank ≈ 1700): the embedding fails to discriminate all 16–30 partitions. A battery of inference-time remedies (subquery MaxSim, finer partitions, fine- and coarse-level overlap,

Table 4. Final production metrics. Pos. is exact-solve rate at the matched half-partition budget $K=10/100/1000$ (Cora/Arxiv/MAG). FilterAll is the exhaustive ceiling. MAG Jigsaw/Mean-RRF use the walk-aware encoder. FeatureIndex is a GraphGrep/gIndex-style label-coverage index, run on positives only.

Dataset	Method	Pos. %	Neg. %	FP	TO	Cand. nodes	Total s	Solver ms
Cora	Jigsaw	94.2	100.0	0	0	2.5K	0.09	7.9
Cora	Mean-RRF	93.8	100.0	0	0	2.4K	0.09	8.3
Cora	MeanFeat	88.8	100.0	0	0	3.5K	0.11	7.4
Cora	TopoFeat	33.4	100.0	0	0	9.5K	0.22	3.2
Cora	Random	31.3	100.0	0	0	9.6K	0.21	3.2
Cora	FilterAll [‡]	100.0	100.0	0	0	0.8K	0.07	11.3
Arxiv	Jigsaw	92.8	100.0	0	0	0.2K	0.59	12.9
Arxiv	Mean-RRF	94.0	100.0	0	0	0.2K	0.41	16.0
Arxiv	MeanFeat	88.9	100.0	0	0	0.2K	0.39	16.5
Arxiv	TopoFeat	33.2	100.0	0	0	0.3K	0.75	5.1
Arxiv	Random	37.7	100.0	0	0	0.3K	0.77	5.4
Arxiv	FilterAll [‡]	100.0	100.0	0	0	0.1K	0.35	16.9
MAG	Jigsaw	88.6	99.8	1	23	138.1K	6.97	81.9
MAG	Mean-RRF	85.4	99.7	1	16	137.1K	5.38	67.9
MAG	FeatureIndex [§]	96.7	—	—	24	59.4K	2.39	118.8
MAG	MeanFeat	64.6	99.8	1	10	212.1K	8.02	27.5
MAG	TopoFeat	37.1	99.8	1	1	235.6K	8.07	5.2
MAG	Random	51.4	99.8	1	8	282.1K	9.54	19.9
MAG	FilterAll [‡]	98.4	99.8	1	30	443.8K	5.85	288.6

Table 5. Cross-dataset label-selectivity crossover: overall median worst-true-partition rank (lower is better; retrieval-only, $n=540$ per dataset). Near-unique feature labels favor the classical index; realistic class labels favor the learned retriever. MAG shows the same inversion at scale.

Dataset (parts)	FeatureIndex Learned Parts/label			
	feature	class	(any)	feat / class
Cora (20)	2	7	5	1 / 12
Arxiv (200)	4	104	44	1 / 124.5

dual-granularity late interaction) does not repair this—best gain $\approx +2.3$ points on the spatial families, far below recovery—because the partition graph is high-degree even at fine granularity (mean fine degree 568 of 10,000); the open issue is representation learning, not aggregation. The crossover itself generalizes: on Cora and Arxiv the winner flips with an offline label-selectivity statistic (median partitions/label 1 at feature labels vs. 12/124.5 at class labels), so Jigsaw selects its bounded retriever by label regime (Table 5). Jigsaw is thus *retriever-agnostic*; the contribution is memory-bounded exact verification.

The central systems result is the scaling break: direct CP-based Glasgow solves all 15 Cora K-hop queries but **0 of 15** on Arxiv and MAG, whereas the

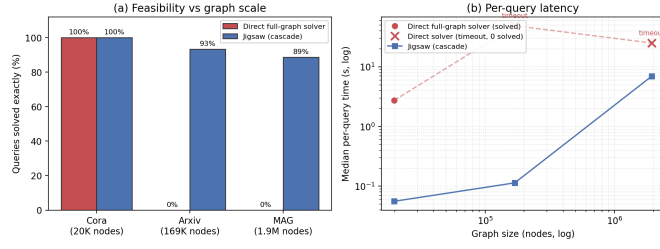


Fig. 2. Jigsaw makes the exact solver’s domain tractable. Direct full-graph Glasgow solves Cora but none of the Arxiv/MAG K-hop samples; Jigsaw stays feasible by verifying only the retrieved-and-pruned candidate.

Table 6. Scaling and latency: direct full-graph solver vs. Jigsaw cascade. Matched K-hop queries; [†] direct solver reaches its 30s budget but Glasgow aborts after cleanup/loading; * MAG Jigsaw is the production mean over all families.

		Direct full-graph solver		Jigsaw cascade		Cand.
Dataset	Nodes	Solved	Med. time	Solved	Med. time	
Cora	19.8K	15/15	2.74 s	15/15	0.06 s	51
Arxiv	169K	0/15 [†]	≈49 s	14/15	0.11 s	50
MAG	1.9M	0/15 [†]	≈25 s	88.6%	6.97 s*	–

Table 7. MAG solve-rate significance. Bootstrap 95% CI over $n=1800$ positive queries, and paired exact-McNemar of Jigsaw vs each policy (wins = queries Jigsaw solves and the other does not).

Method	Solve %	95% CI	vs Jigsaw (W/L, p)
Jigsaw	88.6	[87.1, 90.0]	–
Mean-RRF	85.4	[83.7, 87.0]	92 / 35, $p=4.4 \times 10^{-7}$
MeanFeat	64.6	[62.3, 66.8]	437 / 51, $p < 10^{-76}$
Random	51.4	[49.1, 53.8]	689 / 67, $p < 10^{-130}$
Topo	37.1	[34.9, 39.5]	912 / 32, $p < 10^{-224}$
FilterAll	98.4	[97.8, 98.9]	2 / 225, $p=2.4 \times 10^{-64}$

Jigsaw cascade remains feasible by handing the verifier a small candidate. The retriever therefore sits on a recall-cost frontier: FeatureIndex is strongest under near-unique MAG labels, FilterAll is the exhaustive ceiling, and the learned retriever is a useful memory-bounded point, especially once labels coarsen.

Statistical significance. Table 7 reports bootstrap confidence intervals and paired exact-McNemar tests for MAG positives. Jigsaw significantly beats the learned/feature retrievers except FilterAll, which is the exhaustive ceiling. These are component comparisons; the system contribution is exact matching without whole-graph residence and with fewer candidates than FilterAll.

Table 8. Jigsaw query-family breakdown. Positive columns report exact-solve rate; negative columns report correct no-match rate. All three datasets use the connected-positive protocol. All datasets are reported at the matched half-partition budget ($K=10/100/1000$); the MAG rows use the deployed walk-aware encoder (overall 88.6%).

Dataset	Single	K-hop	Deg-K	Walk	Multi-fine	Multi-coarse	Neg-label	Neg-struct
Cora	100.0	98.3	97.3	75.3	100.0	94.3	100.0	100.0
Arxiv	100.0	97.3	98.3	82.3	100.0	78.7	100.0	100.0
MAG	98.7	93.3	92.7	71.0	100.0	76.0	100.0	99.7

The family view explains the aggregate rates. Single and multi-fine saturation, local K-hop families remain high, and random-walk/multi-coarse are hardest across datasets. MAG still solves single 98.7%, K-hop 93.3%, degree K-hop 92.7%, and multi-fine 100%; random walks remain lowest at 71.0%.

Random walks expose the main failure. FilterAll solves them at 96.0%, and 96.6% of Jigsaw’s random-walk misses occur before solving, so the issue is retrieval/overlap coverage, not Glasgow. Compact K-hop misses often lie on retrieved boundaries and overlap repairs them; diffuse walk misses often lie in unretrieved interiors. The held-out walk-aware encoder lifts random-walk 60.0% $\rightarrow$ 71.0% and multi-coarse 71.7% $\rightarrow$ 76.0% relative to the generic encoder, but a single-vector query embedding still struggles with spatially spread matches.

Cumulative solved-by-budget curves rise monotonically to 94.2/92.8/88.6 at the half-partition budget. On MAG, solve rate rises from 37.6% at small budgets to 88.6% at $K=1000$, so failures are insufficient candidate expansion, not verifier unsoundness.

6.3 Candidate-Shrinkage Cascade and Selective Overlap

A natural objection is that one-hop overlap might expand to most of the graph. We test the per-query cascade

full graph $\rightarrow$ overlap $\rightarrow$ signature $\rightarrow$ label-pruned $\rightarrow$ solver components,

on 14,400 MAG queries (10,800 positives). Three facts emerge.

(1) Overlap is broad, but ranking still removes most of the graph. The median selected-overlap union is 59% of MAG for Jigsaw, yet after identical pruning the candidate is 84.5K nodes versus 544K for FilterAll – a $6.4\times$ median reduction.

(2) Pruning is coverage-lossless. Recall loss occurs at retrieval/overlap, not attribute pruning:

Proposition 1 (Lossless pruning). *Let the pruning token $\sigma(\cdot)$ be a node attribute that the matching semantics preserve, i.e. any valid embedding ϕ of Q into G satisfies $\sigma(\phi(q)) = \sigma(q)$ for every query node q (vertex-labelled matching,*

with σ derived from node type and feature labels). Prune the candidate G_C to $G'_C = \{v \in G_C : \sigma(v) \in \sigma(V_Q)\}$. Then for any embedding ϕ with $\phi(V_Q) \subseteq G_C$, we have $\phi(V_Q) \subseteq G'_C$. Hence pruning preserves node coverage: FullCov is unchanged.

Proof. For $q \in V_Q$, $\sigma(\phi(q)) = \sigma(q) \in \sigma(V_Q)$, so $\phi(q)$ satisfies the keep condition and is retained. Thus every matched node survives. $\square$

The cascade signatures and exact-label filters are attribute invariants, so the proposition applies. Empirically, planted-match survival after overlap equals survival after pruning for every policy (Jigsaw 87% both), so the solve ceiling is set by whether the retrieved-plus-overlapped region contains the match.

(3) Candidate construction dominates latency: mean build is 3.4 s for Jigsaw and 5.5 s for FilterAll versus 0.25 s of solving.

Why broad overlap is mostly wasted, and the selective-overlap fix. One-hop overlap often inflates candidates without repairing misses because it reaches boundary nodes, not interiors of fully unselected partitions. Selective overlap therefore keeps only high-support or label-compatible neighboring boundaries. In the probe, strongest-neighbor capping cuts the overlap union by $\approx 3.1\times$, label-compatible expansion by $\approx 5.0\times$, and both by over $6\times$, exposing the main latency lever.

6.4 What the Ablations Establish

The ablations show why the cascade needs multiple operators. FullCov improves retrieval coverage (Table 3). On MAG, removing overlap halves positive solves ($88.9 \rightarrow 44.4\%$); on Arxiv, removing signatures inflates candidates $\sim 75\times$ ($94 \rightarrow 7,081$ nodes). Exact-label pruning matters for candidate size and negative rejection (MAG solve $88.9 \rightarrow 52.8\%$), and component decomposition helps MAG ($88.9 \rightarrow 63.9\%$). No single operator is universally idle.

Diagnostic variants are kept separate from the canonical validation-selected matrix.

6.5 Negative Queries and Soundness

Jigsaw never accepts from the neural model alone; Glasgow is the only accepting step. Audited no-match accuracy is 100.0% for Cora/Arxiv and 99.7–99.8% for selected MAG rows. The rare false positives are benchmark-level failures, not verifier hallucinations: a returned mapping is a true isomorphism inside the candidate, but a perturbed “negative” may still match elsewhere. Globally certified negatives remain future work.

7 Discussion and Limitations

Jigsaw is an exact-verification system with a learned boundary, not a proof of global completeness: if retrieval misses a necessary region, the solver cannot recover the planted match. It is strongest for static attributed graphs and repeated

query workloads where partitioning, embeddings, and FAISS indexes amortize. FeatureIndex is strongest under near-unique labels; learned retrieval becomes the better localizer as labels coarsen.

The main limitation is comparison scope. We compare against feature/structure index rankers, no-retrieval Glasgow, exhaustive FilterAll, and uninformed retrievers, but not learned node-alignment systems such as NeuroMatch or GLSearch, which target smaller contained queries. Scalable filter-and-verify matchers solve our graphs once loaded, so the infeasibility claim is scoped to CP-based Glasgow and memory/low-selectivity regimes. Out-of-core and distributed matchers also avoid residence; Jigsaw instead trades offline cost for bounded-memory serving. Finally, the benchmark uses planted connected subgraphs on static label-rich graphs, so recall-decisive learning needs harder weak-label or non-planted workloads.

8 Conclusion

Jigsaw connects learned retrieval and exact verification through a measurable coverage condition: FullCov improves the probability that all required regions are retrieved, and overlap and pruning make exact verification tractable – not replacing exact solvers but making them usable at scale under bounded memory. Jigsaw is *retriever-agnostic*: which retriever wins depends on label selectivity, and Jigsaw exploits this by selecting its bounded retriever from that offline signal—a classical index under near-unique labels, the learned retriever under realistic ones (confirmed on Cora, Arxiv, and MAG). Making retrieval decisive on spatially-extended queries stays open: inference-time remedies at both partition granularities did not recover coverage, pointing to coverage/footprint-aware representation learning rather than inference-time aggregation.

Use of Generative AI. Generative AI tools assisted with language editing and software scaffolding; all research design, experiments, analyses, and claims are the authors’ own.

References

1. McCreesh, C., Prosser, P., Trimble, J.: The Glasgow Subgraph Solver: Using Constraint Programming to Tackle Hard Subgraph Isomorphism Problem Variants. In: CP 2020 (2020)
2. Xu, K., Hu, W., Leskovec, J., Jegelka, S.: How Powerful are Graph Neural Networks? In: ICLR (2019)
3. Cordella, L.P., Foggia, P., Sansone, C., Vento, M.: A (Sub)Graph Isomorphism Algorithm for Matching Large Graphs. IEEE TPAMI **26**(10), 1367–1372 (2004)
4. Johnson, J., Douze, M., Jégou, H.: Billion-Scale Similarity Search with GPUs. IEEE Trans. Big Data **7**(2), 535–547 (2021)
5. Karypis, G., Kumar, V.: A Fast and High Quality Multilevel Scheme for Partitioning Irregular Graphs. SIAM J. Sci. Comput. **20**(1), 359–392 (1998)

6. Hu, W., et al.: Open Graph Benchmark: Datasets for Machine Learning on Graphs. In: NeurIPS (2020)
7. Garey, M.R., Johnson, D.S.: Computers and Intractability: A Guide to the Theory of NP-Completeness. W.H. Freeman (1979)
8. Bonnici, V., Giugno, R., Pulvirenti, A., Shasha, D., Ferro, A.: A Subgraph Isomorphism Algorithm and Its Application to Biochemical Data. BMC Bioinformatics **14**(Suppl 7), S13 (2013)
9. Solnon, C.: AllDifferent-Based Filtering for Subgraph Isomorphism. Artificial Intelligence **174**(12–13), 850–864 (2010)
10. Sen, P., Namata, G., Bilgic, M., Getoor, L., Galligher, B., Eliassi-Rad, T.: Collective Classification in Network Data. AI Magazine **29**(3), 93–106 (2008)
11. Ying, R., et al.: Neural Subgraph Matching. arXiv:2007.03092 (2020)
12. Ye, Y., Lian, X., Chen, M.: Efficient Exact Subgraph Matching via GNN-based Path Dominance Embedding. Proc. VLDB Endow. **17**(7), 1628–1641 (2024)
13. Liu, Y., et al.: GLSearch: Maximum Common Subgraph Detection via Learning to Search. In: ICML (2021)
14. Liu, X., Pan, H., He, M., Song, Y., Jiang, X., Shang, L.: Neural Subgraph Isomorphism Counting. In: KDD (2020)
15. Han, W.S., Lee, J., Lee, J.H.: TurboISO: Towards Ultrafast and Robust Subgraph Isomorphism Search in Large Graph Databases. In: SIGMOD (2013)
16. Bi, F., Chang, L., Lin, X., Qin, L., Zhang, W.: Efficient Subgraph Matching by Postponing Cartesian Products. In: SIGMOD (2016)
17. Han, M., Kim, H., Gu, G., Park, K., Han, W.S.: Efficient Subgraph Matching: Harmonizing Dynamic Programming, Adaptive Matching Order, and Failing Set Together. In: SIGMOD (2019)
18. Carletti, V., Foggia, P., Saggese, A., Vento, M.: Challenging the Time Complexity of Exact Subgraph Isomorphism for Huge and Dense Graphs with VF3. IEEE Trans. Pattern Anal. Mach. Intell. **40**(4), 804–818 (2018)
19. Zeng, L., Zou, L., Özsu, M.T., Hu, L., Zhang, F.: GSI: GPU-friendly Subgraph Isomorphism. In: ICDE (2020)
20. Sun, S., Luo, Q.: In-Memory Subgraph Matching: An In-depth Study. In: SIGMOD (2020)
21. Kim, H., Lee, J., Bhowmick, S.S., Han, W.-S., Lee, J., Ko, S., Jarrah, M.H.A.: DUALSIM: Parallel Subgraph Enumeration in a Massive Graph on a Single Machine. In: SIGMOD (2016)
22. Sun, Z., Wang, H., Wang, H., Shao, B., Li, J.: Efficient Subgraph Matching on Billion Node Graphs. Proc. VLDB Endow. **5**(9), 788–799 (2012)
23. Yan, D., Guo, G., Chowdhury, M.M.R., Özsu, M.T., Ku, W.-S., Lui, J.C.S.: G-thinker: A Distributed Framework for Finding Qualified Subgraphs in a Big Graph with Load Balancing. In: ICDE (2020)